# Influence Functions Replication Report

Koh & Liang, Figure 2 (Middle Panel)

Zhe Fu

April 15, 2026

## TL;DR

I trained an L2-regularized logistic regression on MNIST ($n = 55000$), implemented both influence-function algorithms from the paper (CG for the exact solution and LiSSA for the stochastic approximation), and verified the predictions against leave-one-out retraining as ground truth. The final numbers are Pearson = **0.9983**, Spearman = **0.9890**, sign match = **1.0000**, which matches the middle panel of Figure 2 in the paper.

Along the way I hit three bugs: (1) wrong effective regularization in LiSSA, (2) too few CG iterations, and (3) float32 precision issues in the LOO retraining. Details below.

## 1 What I'm replicating

The paper's core formula: estimate "how much does the test loss at $z_{\text{test}}$ change if I upweight training point $z$ slightly?" Their answer is:

$$\mathcal{I}_{\text{up,loss}}(z, z_{\text{test}}) = -\nabla_\theta L(z_{\text{test}})^\top H_{\hat{\theta}}^{-1} \nabla_\theta L(z)$$

Directly computing $H^{-1}$ is infeasible here (parameter dimension $d = 784 \times 10 = 7840$), so the approach is to first compute

$$s_{\text{test}} = H_{\hat{\theta}}^{-1} \nabla_\theta L(z_{\text{test}})$$

and then take one inner product per training point: $-s_{\text{test}} \cdot \nabla L(z)$. The hard part is computing $s_{\text{test}}$, and the paper gives two methods — I implemented both.

**What Figure 2 middle panel shows**: the x-axis is the true LOO loss difference (remove one training point, retrain, measure how the test-loss at $z_{\text{test}}$ changed), and the y-axis is the LiSSA stochastic approximation's predicted difference. If both methods are correct, the points should lie on the line $y = x$.

## 2 Pipeline

```
train_linear_mnist.py              train baseline
         |
         v
inspect_test_point.py              choose a test point (idx=8, misclassified)
         |
         +------> cg_inverse_hvp.py              (exact solution)
         |              |
         |              v
         |       s_test_cg_idx8.pt
         |              |
         +------> stochastic_inverse_hvp.py      (LiSSA approximation)
                        |
```

```
                v
      s_test_idx8_r10_t5000.pt
                |
                v
compute_predicted_influence.py   compute prediction for each train point
                |
                v
loo_retrain_topk.py              LOO retraining for ground truth
                |
                v
     Figure 2 middle plot
```

| Script | Role |
|---|---|
| `influence_utils.py` | Shared utilities (model class, HVP, gradients, losses) |
| `train_linear_mnist.py` | Train the baseline linear classifier |
| `hvp_sanity_check.py` | Verify HVP implementation by linearity |
| `inspect_test_point.py` | Pick a misclassified test point |
| `cg_inverse_hvp.py` | Conjugate Gradient for $s_{\text{test}}$ (exact) |
| `stochastic_inverse_hvp.py` | LiSSA for $s_{\text{test}}$ (approximation) |
| `compute_predicted_influence.py` | Predicted loss-diff for every training point |
| `loo_retrain_topk.py` | Top-K leave-one-out retraining + scatter plot |

# 3 Three bugs I ran into

## Bug 1: Wrong effective regularization in LiSSA (the critical one)

The LiSSA recursion is

$$h_j = v + (1 - \text{damping})\, h_{j-1} - \frac{H_z}{\text{scale}}\, h_{j-1}$$

which in expectation converges to $(H + \text{damping} \cdot \text{scale} \cdot I)^{-1} v$. So the **actual extra regularization added** is damping $\times$ scale, not damping alone.

I initially used values from some reference implementations: damping=0.01, scale=50. That gave an effective extra regularization of $0.01 \times 50 = 0.5$, which is **50**$\times$ larger than my real L2 regularization of 0.01. This effectively solves a different linear system, so the stochastic approximation and the CG solution disagreed significantly.

**Fix**: since L2 = 0.01 already makes the Hessian strictly positive definite, no extra damping is needed. I set damping = 0 and scale = 25 (scale is there to ensure $\|H_z/\text{scale}\| < 1$ so the recursion is a contraction).

## Bug 2: Too few CG iterations

Parameter dimension is 7840. My initial `MAX_CG_ITERS=30` was far too small. CG is guaranteed to converge in at most $d$ steps in exact arithmetic, and in practice the L2-regularized Hessian has a decent condition number, so 200 iterations is enough to hit tol = 1e-8.

**Fix**: raised `MAX_CG_ITERS` from 30 to 200, with early stopping on residual norm.

## Bug 3: LOO precision issues (the trickiest one)

In LOO retraining we want the change in test loss after removing one point, which is typically in the range $10^{-3}$ to $10^{-5}$. With float32 + L-BFGS warm-started from the full-data solution, many points gave an "actual diff" of exactly 0.0 (producing a horizontal band of points at $x = 0$ in the scatter plot).

Two issues stacked on top of each other:

1. **Float32 isn't precise enough**: the warm-start is already within $\sim 10^{-5}$ of the optimum. The quadratic model L-BFGS builds lives inside that gap, and in float32 the line-search step sizes get rounded to zero — the optimizer simply doesn't move.
2. **Default tolerance is too loose**: PyTorch's L-BFGS defaults to `tolerance_grad=1e-7`, but the warm-start's initial gradient has magnitude $O(1/n) \approx 1.8 \times 10^{-5}$, so the stopping criterion triggers at iteration 0 — it returns the warm-start unchanged.

**Fix**: float64 + CPU (MPS doesn't support float64) + `tolerance_grad=0` + `tolerance_change=0` + `max_iter=200` + strong Wolfe line search. After this, Pearson jumped from quite poor to 0.998.

# 4 Final results

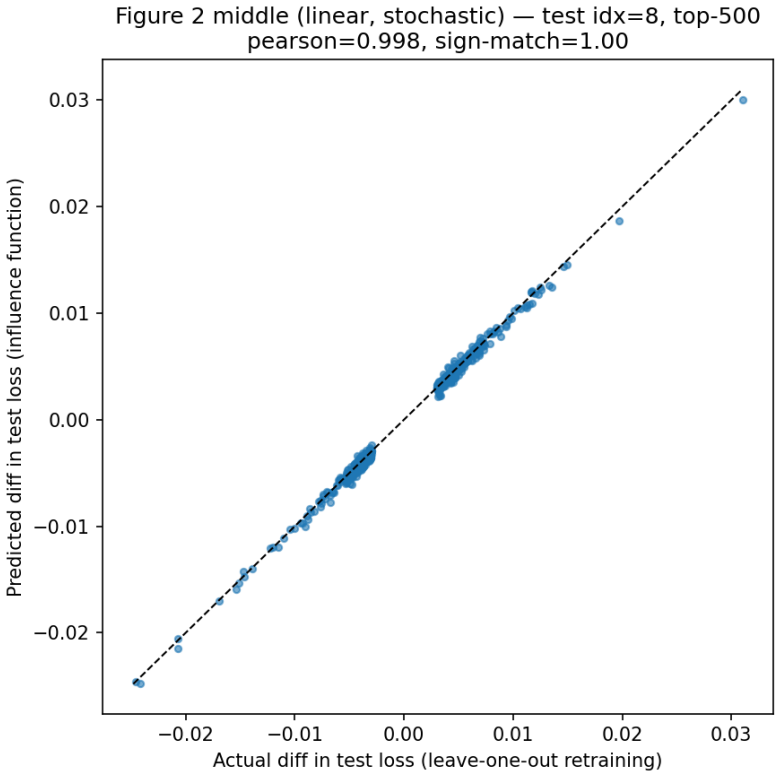


Figure 1: Replication of Figure 2 middle panel. x-axis: actual LOO test-loss difference. y-axis: LiSSA predicted test-loss difference.

| Metric | Value |
|---|---|
| Pearson correlation | 0.9983 |
| Spearman correlation | 0.9890 |
| Same-sign ratio | 1.0000 |
| Top-K | 500 (selected by \|influence\| from CG) |
| Test point | idx = 8, true label 5, model predicts 6 |

Following the Figure 2 middle-panel protocol: I selected the top-500 training points by |CG influence|, and then plotted LiSSA's predictions for those same 500 points on the y-axis. Keeping selection and prediction on separate pipelines prevents stochastic noise from biasing which points get chosen.

## 5 What I understood, and what I didn't

**What I understood**:

- Why we use HVPs instead of materializing the Hessian explicitly ($d \times d$ memory blowup).
- Pearlmutter's trick: differentiating the scalar $\nabla L \cdot v$ again gives $Hv$ — all with standard autograd.
- Why CG works well on positive-definite matrices, and how L2 regularization effectively acts as preconditioning here.
- Why LiSSA needs the `scale` factor (so that $\|H_z/\text{scale}\| < 1$, making the update a contraction).

**What I'm not sure about yet**:

- How influence functions behave on non-convex models (deep networks) where the Hessian isn't PSD. The paper discusses this, but I haven't gone deep yet.
- Convergence theory for LiSSA in high-dimensional, non-convex settings.
- Why scale = 25 is enough in practice. I used a rough upper bound from MNIST input norms, but a tighter bound would need more careful analysis.

## 6 Next steps

- Try the CNN case (Figure 2, right panel).
- Look into how to handle negative eigenvalues in the non-convex setting.
- Explore influence functions for dataset cleaning / mislabel detection.

## A Key hyperparameters

- Training: $n = 55000$, `L2_REG = 0.01`, L-BFGS to convergence.
- Test point: idx = 8 (a misclassified 5 — model predicts 6).
- CG: `max_iter = 200`, `tol = 1e-8`.
- LiSSA: $R = 10$ repetitions, $T = 5000$ steps, scale = 25, damping = 0.
- LOO: top-500, float64, CPU, L-BFGS with zero tolerances, `max_iter = 200`, warm-start.

## B AI assistance disclosure

I used Claude to help with:

- Tracking down the three bugs above — especially the LiSSA effective-regularization one, which was diagnosed by computing the fixed point of the recursion.
- Sanity-checking my HVP implementation using a linearity test.
- Debugging the float32 / tolerance issue in LOO retraining (working backwards from the horizontal-band symptom).
- Walking through parts of the paper's math and organizing my thinking.
- Brainstorming code structure.

My own work: understanding the paper, all architecture and implementation decisions, analyzing the results, and writing this report.